

Bruce Rick

Ottawa, Canada

Email: brucericktgs@gmail.com

Website: <https://brucerick.github.io/me/>

Experience

theScore / Penn Entertainment | iOS Developer
2022-2025

Apps: theScore | theScore Bet

Skills: Swift | SwiftUI | Concurrency | IGListKit | REST | GraphQL | Dependency Injection | GitHub | GitHub Actions | Snapshot Testing | Nimble | Quick | Figma

Content Team | theScore

- **theScore.** Sports news app available on iOS app store. I was a part of the content team responsible for maintaining the news feed lists.
- I was responsible for the News Feed redesign, social reactions and clips features.
- theScore used View Controllers in combinations with Coordinators and View Models in its own custom architecture.
- News Feed used **IGListKit** library originally to create table sections of reusable table view cells to create news feed 'cards.' I was responsible for a UI Redesign which called for updating **UIKit** table view cells and snapshot tests. Eventually we began converting sections to **SwiftUI** and using Hosting Cells to display them in **CollectionKit**.
- I was responsible for the feature of letting users react to news feed cards and articles. I used **REST** and eventually **GraphQL** to grab reaction data and send user reactions to our backend.
- I was responsible for the clips feature. On the matchup page, video clips were auto ingested from Twitter and video files were displayed in a view like instagram reels.
- I was part of the **SwiftUI** knowledge transfer team. Early on, theScore was behind in adopting **SwiftUI**. I was a part of a team to help educate other developers on how to use it. I ran demo sessions and knowledge transfer sessions. As well as helping other developers when issues arose when using **SwiftUI**.
- I used various **dependency injection** systems for all features from World singleton to injected repository and client classes. These dependencies were overridden in tests to ensure no production API calls or untestable code was run.
- We used our own in-house reactive code system but eventually migrated to combine and concurrency.
- I extensively used **Snapshot Testing** when building my features. This ensured that all UI changes were tracked and flagged. It involved creating a view and taking a snapshot image and storing it, running a comparison tool to ensure the image was the same vs the previously committed code and image. Nimble and Quick as well as a snapshot

library were used for testing. Before code could be merged, it had to be 90% code coverage tested.

- I used GitHub to manage our code base and perform reviews and code merges.
- I used GitHub actions to maintain our code base and perform actions like running tests and uploading code to the app store.
- I extensively used **Figma** to view concept files from designers and get all margin values / colour values / layouts / animations to recreate designs exactly.

AlleyOop Team | theScore | theScore Bet | ESPN Bet | BarStool Bet

- theScore Bet. Sports betting app available on iOS app store. I was a part of the design system team that created UI elements to be used and reskinned by the app in its various forms (BarStool Bet / ESPN Bet / theScore Bet US).
- I was involved with creating all kinds of UI components to be used across both the news app as well as the betting apps from without our internal design library AlleyOop.
- Created Carousel / Chip / Badge / TextScrollFade and other UI components.
- Worked extensively with **Figma** APIs to autogenerate colour / margin / spacing value code. Also uploaded code snippets so developers can see AlleyOop code and usage right from within a Figma design file.
- I had to work with feature teams to use our design library to recreate the designs created by feature team designers. This involves hands-on training or debugging with specific developers or teams.
- I had to work with a very large team to navigate my features. This included proper dependency injection across all features for consistency, snapshot testing to ensure that any UI changes were caught by visual screenshots before any code changes were merged as well as constant knowledge transfer sessions between my 100s of teammates and me.

Lixar I.T. | Senior iOS Developer
2020-2022

Clients: Government of Canada | CBSA

Apps: ELVIS

Skills: Swift | SwiftUI | Combine | Swift Composable Architecture | REST | AWS | AWS Dev Ops | Fastlane | Nimble | Quick | Adobe XD

ELVIS | Ecommerce Low Value Inspection System

- Internal iOS app for **CBSA** agents to scan and perform inspections on packages traveling into and outside Canada.
- I was responsible for the flight inspection screen and various UI elements like header and footer bar.
- App used **SwiftUI** version 1 along with the **Swift Composable Architecture**

- I had to learn **SwiftUI** extensively from ground up and understand the concepts of reactive programming. Used the concept of reducers and effects in **Swift Composable Architecture** to break down and organize **Combine** framework calls into better organized and safer code.
- Had to extensively use **UIKit** views in a **SwiftUI** architecture via Hosting Controllers to perform certain actions that weren't initially available in **SwiftUI**.
- App performed **REST** API calls via Apples **URLSession**.
- I was involved in the entire development cycle of existing and new apps. Planning, estimating, developing, testing, releasing, and bug fixing.
- I was involved at a high level in all levels of app development. Backend, continuous integration, app submission for release, and quality assurance.
- I collaborated and presented features to clients and the entire development team.
- I had business trips to client offices to estimate or get feedback on new features.
- I gave feedback and developed a consistent architecture to maintain project requirements and testability with other iOS developers.
- I solved complex problems by developing, testing, implementing, and explaining solutions based on client and user feedback.
- **Amazon Web Services** was used for project management. **AWS Dev Ops** was used for continuous integration, running tests before releases and after every code merge. **AWS** hosted our GIT repo and was used for pull requests and code reviews. I maintained and used Fastlane via **AWS Dev ops** plans to perform various actions on the code base like running tests, configuring code for release and uploading build files to app store.
- I used **XCTest** in combination with the **Nimble** and **Quick** to perform tests within the Swift Composable Architecture system.
- **Adobe XD** was used by our designers to create mockups which then I used to create layouts in code that matched exactly.

Lixar I.T. | Intermediate iOS Developer
2015-2020

Clients: Delphi Automotive | Allegiant Airlines | AirEuropa

Apps: FlyDesk | Allegiant | CrewPad

Skills: Swift | XCTest | Realm | ReactiveSwift | REST | GraphQL | Swagger | OpenAPI | JIRA | BitBucket | TeamCity | Fastlane | Zeplin

CrewPad

Swift | **XCTest** | **Realm** | **ReactiveSwift** | **REST** | **Swagger** | **OpenAPI** | **JIRA** | **BitBucket** | **TeamCity** | **Fastlane** | **Zeplin**

- iPad app for cabin crew for the European airline **AirEuropa**. Flight attendants could do pre-flight check lists, manage inflight purchases, passenger manifests and report flight incidents via app.
- I was responsible for flight reports and flight messages features as well as fixing bugs.

- **Agile and scrum** development style. Two-week sprint cycles with releases. Daily standups with sprint start planning sessions, end of sprint demos and retrospectives.
- App was built in **Swift** with **UIKit** and constraint / storyboard system.
- Used a View Controller architecture but used **ReactiveSwift** to bind ViewModels to **UIKit**. This meant no longer needing delegate calls every time UI or data is updated – we binded data to a UI element via signals like what eventually came via SwiftUI.
- Our database was managed via third party library '**Realm**.' This system was more user friendly than Apples **CoreData** system however it required larger attention to detail with thread safety and bugs relating to user actions. Had to overcome challenges when adopting this with **ReactiveSwift** – had a lot of completion blocks that had to be meticulously managed and tested to remove memory leaks and properly make realm database changes to maintain thread safety.
- Download specific flight crew user data via **REST** API calls. I used the standard Apple framework to make and decode **JSON REST** responses into model objects. For this App I used **Swagger / OpenAPI** YAML files to create REST templates. These templates specified a template for our API requests, responses and data structure. These templates were then used to auto generate boilerplate code for both the iPad app and the backend. This meant there was no disconnect between client and backend service (such as misnamed JSON values and URLs or incorrect object structure) as we worked via the same template file.
- **JIRA** was used for project management. **TeamCity** was used for continuous integration, running tests before releases and after every code merge. **BitBucket** hosted our GIT repo and was used for pull requests and code reviews. I maintained and used Fastlane via **Teamcity** plans to perform various actions on the code base like running tests, configuring code for release and uploading build files to app store.
- **Zepplin** was used by our designers to create mockups which then I used to create layouts in code that matched exactly.
- **XCTests** to perform unit tests as well as UI Tests. Our target coverage was 90% code coverage.

Allegiant consumer app

Swift | REST | GraphQL | JIRA | BitBucket | TeamCity | Fastlane | Zepplin

- Consumer iOS app available on iOS app store. Customers can review flight details, check into flights and receive boarding passes, view account details, or purchase flights and other vacation services.
- I was responsible for Flight timeline feature and user management redesign features.
- **Agile and scrum** development style. Two-week sprint cycles with releases. Daily standups with sprint start planning sessions, end of sprint demos and retrospectives.
- Used a ViewController architecture with constraints and storyboard systems.
- The timeline feature was a system of cards in a timeline like the Facebook timeline that showed the users booked flights. Users were able to see arrival and departure times and do things via these cards like check-in online, pay for packages, upgrade services and

view boarding passes. I was responsible for the creation of this feature from start to release. Other features included dynamic sizing cards that could be expanded and collapsed, and fetching data via **REST** calls.

- App used **RESTKit** originally but I converted to remove third party library and rely solely on Apple **URLSession** libraries.
- Plans to do a complete redesign of the app for all other areas outside the timeline began and I flew to Allegiant headquarters in Las Vegas to plan out features and take part in estimates for features.
- Implemented initial version of user redesign of the user profile section, this began to upgrade from **REST** calls to **GraphQL** calls made via the **Apollo** iOS library.
- **JIRA** was used for project management. **TeamCity** was used for continuous integration, running tests before releases and after every code merge. **BitBucket** hosted our GIT repo and was used for pull requests and code reviews. I maintained and used Fastlane via **Teamcity** plans to perform various actions on the code base like running tests, configuring code for release and uploading build files to app store.
- **Zepplin** was used by our designers to create mockups which then I used to create layouts in code that matched exactly.
- **XCTests** to perform unit tests as well as UI Tests. I specifically created some tests around boarding passes. We had test coverage that didn't properly test user facing features relating to boarding passes. We had a release where boarding pass feature was broken and not caught by tests. Created a UI test that ensured boarding pass opened from various screens and displayed boarding pass and QR code as desired.

FlyDesk

Swift | REST | CoreData | PSPDFKit | JIRA | BitBucket | TeamCity | Fastlane | Zepplin

- In-house iPad app intended for pilots and maintenance crew to view PDF manuals for training, perform reports and view airline notices.
- I was responsible for creating a login system and separating the stored databases per user.
- **Agile and scrum** development style. Two-week sprint cycles with releases. Daily standups with sprint start planning sessions, end of sprint demos and retrospectives.
- App used a login system that returned a **JWT** token to be stored and used to perform **REST API** calls. Had to create the login page as well as the re-authentication view when the token expires. Data was specific and sensitive to each user, and I was responsible for creating a system where entirely separate database files were created per user to maintain the apps' existing singular user structure.
- App was built upon the same code base as InFlight and used **CoreData** to store user data.
- A core feature of the app was using **PSPDFKit** so users could view downloaded PDFs via **REST to view on iPad**. I was responsible for storing user generated Bookmarks in the User table.

- **JIRA** was used for project management. **TeamCity** was used for continuous integration, running tests before releases and after every code merge. **BitBucket** hosted our GIT repo and was used for pull requests and code reviews. I maintained and used Fastlane via **Teamcity** plans to perform various actions on the code base like running tests, configuring code for release and uploading build files to app store.
- **Zepplin** was used by our designers to create mockups which then I used to create layouts in code that matched exactly.

Lixar I.T. | Junior iOS Developer
2013-2015

Clients: Delphi Automotive | Allegiant Airlines

Apps: Delphi Connect | InFlight.

Skills: Objective-C | Swift | REST | CoreData | Payment Peripherals | HTML | Localization | JIRA | BitBucket | Bamboo

InFlight

Objective-C | Swift | REST | CoreData | Payment Peripherals | JIRA | BitBucket | Bamboo

- In-house iPad app intended for flight attendants to sell goods and services during travel.
- I was responsible for maintaining localization, a **CoreData** database with migrations, and using payment peripherals to process customer payments aboard a flight.
- Maintained the ability for the app to be localized and translated.
- **Agile and scrum** development style. Two-week sprint cycles with releases. Daily standups with sprint start planning sessions, end of sprint demos and retrospectives.
- InFlight had a user log in for each flight attendant which fetched user specific data via **REST** API calls and stored in a **CoreData** database.
- Maintained **CoreData** through **XCodes CoreData** IDE. Had to add and modify existing databases for implementation of new features. Ensuring no database conflicts or deletions when new app versions were released was critical and maintained through Apples **CoreData** migrations IDE. I was responsible for creating a test which had saved versions of the database for every app release and attempted to migrate to the current version. This test ensured that **CoreData** items created in version 1 were still available and properly stored in the latest release.
- I had to maintain an in-house 3rd party library that detected various payment peripherals plugged into the iPad and initialized various 3rd party SDKs pertaining to the correct connected peripheral. Fixed issues around SDKs needing to be dynamically started and stopped when peripherals were connected and disconnected as opposed to on app start and finish as instructed.

Delphi Connect

Objective-C | Swift | REST | HTML | Localization | JIRA | BitBucket | Bamboo

- App that received data and sent commands to a device plugged into OBD port of vehicle.
- I was responsible for the **Localization, Theming, Swift Conversion, and Eco-Drive** features.
- **Agile and scrum** development style. Two-week sprint cycles with releases. Daily standups with sprint start planning sessions, end of sprint demos and retrospectives.
- **Localization** - Localized app for other languages. Created localization files with tokens and maintained translations via a webservice. Helped with 3rd party integration through **Bamboo** to automatically import and export excel files to auto update localization files.
- **REST** - Restful services to fetch API data via a 3rd party library (**RESTKit**) and store on device such as vehicle and user details.
- **JIRA | BitBucket | Bamboo** - I used the **JIRA** for project management, estimating features and tracking issues. **BitBucket** to host our code and perform code reviews. **Bamboo** as our continuous integration system - automatically running tests and creating build files for distribution.
- **UIKit | HTML Templates | Objective-C** - code base was written in **Objective-C** in the **UIKit** framework. I used HTML templates for main screen UI to share UI code between Android and iOS via a **HTML** rendering engine called 'Velocity.' Used a ViewController architecture with an embedded WebView to display templates.
- I was the lead developer for the Eco-Drive feature. A screen that fetched user driving statistics relating to driving habits via **REST** calls. Built in **Swift** with native UIKit support for graphs and animations.
- Responsible for theming rebranding. Maintaining a theming engine that hosts multiple **CSS** files for various clients to reskin app.
- Responsible for **Swift** conversion. We eventually moved away from HTML templates to native screens using **Swift**. This meant converting **Objective-C** view controllers embedding **HTML** templates to full swift view controllers using **UIKit** and constraints to natively display data.

Conferences

- **WWDC 2024**. I attended with 3 other colleagues to represent theScore. Attended the 'One More Thing' conference from Paul Hudson during the same week. I learned about Apple Intelligence, new Apple features. Spent time speaking to Apple Developers and brought back some knowledge about how to improve various systems at theScore.
- **Try Swift! Tokyo**. I learned about additions to Apple localization systems. Became familiar with the Apple developer community in Japan. I got hands on experience from Point Free about Swift Composable Architecture and the new addition of SQLite Data to the framework.
- **Try Swift! Toronto**. I learned about new design methods and supported colleagues from theScore who were presenting.

Education

Algonquin College | Ontario College Advanced Diploma | Game Development

2010-2013

Game Development | C++ | C# | Objective-C | XNA | Unity | Adobe Creative Suite | Autodesk 3DS Max

- Instructed in all aspects of game development.
- Learned to program in many different programming languages for general game logic and 3D/2D rendering.
- Used various frameworks, IDEs and game engines to create 2D and 3D games on PC/Xbox/iPhone platforms.
- Learned to create 3D art with modelling software tools.
- Learned to create 2D art with photo editing and vector software.
- Learned about game design theory and principles.
- Was given English and mathematical education as it pertains to game development.
- Had general electives outside of the scope of game development which included ethics and community service.